



EXAMEN DE LA FEDE

**Rapport d'activité présenté à l'épreuve professionnelle de la soutenance du
Bachelor Européen Développement d'Applications et Solutions Connectées de
l'année 2026**

BESTMARK — Développement d'une application web de gestion de magasin en ligne

Raison sociale : BESTMARK — Projet tutoré (application fictive, réalisé au sein de l'école)

Adresse de l'entreprise : — (projet tutoré, sans entreprise d'accueil)

Nom : HASNAOUI

Prénom : Aimad

Numéro du candidat : 165734

Nom et prénom du tuteur enseignant : **Mr. Abdelmajid BOUHO**

Année scolaire : **2025/2026**

SOMMAIRE

I. Introduction et présentation du projet

2

- 1. Contexte et enjeux 2
- 2. Objectifs de la mission 3
- 3. Pile technologique 3
- 4. Architecture de haut niveau 4
- 5. Périmètre et limites du projet 4
- 6. Public cible et utilisateurs de l'application 5

II. Méthodologie de travail

5

- 1. Outil de suivi : Notion 5
- 2. Organisation du développement 6
- 3. Planning prévisionnel 6
- 4. Environnement de travail et outils 7

III. Description de la mission — développement par module métier

8

- 1. Gestion des produits et catégories 8
- 2. Gestion des fournisseurs 9
- 3. Gestion des achats (réapprovisionnement) 10
- 4. Gestion du stock (mouvements et traçabilité) 10
- 5. Gestion des ventes et de la facturation 11
- 6. Gestion des livraisons 12
- 7. Gestion des demandes clients (liste d'attente) 13
- 8. Gestion des employés et de la paie 14
- 9. Suivi financier centralisé (dépenses et transactions) 14

IV. Difficultés rencontrées et compétences acquises

15

- 1. Sécurisation de l'API 16
- 2. Gestion centralisée des erreurs 16
- 3. Modélisation des relations de données complexes 16

4. Organisation du code par domaine métier 17	
5. Compétences acquises 17	
V. Tests et validation	18
1. Tests de l'API au fur et à mesure du développement 18	
2. Scénarios de bout en bout testés manuellement 18	
3. Vérification de la sécurité de base 19	
4. Bilan des tests et limites de l'approche manuelle 19	
VI. Structure du dépôt	20
VII. Backend — API REST	21
1. Cycle de vie du serveur 21	
2. Table des endpoints REST 22	
3. Modèles de données Mongoose 23	
4. Authentification — middleware Protect 25	
5. Gestion des erreurs asynchrones 26	
6. Variables d'environnement 27	
VIII. Frontend Web	27
1. Point d'entrée et bootstrap d'authentification 27	
2. Table des routes 28	
3. Couche API — services et intercepteurs Axios 28	
4. Organisation des écrans par domaine 29	
5. Écran de vente — formulaire composite 29	
IX. Flux de données — scénarios clés	30
1. Connexion d'un employé 30	
2. Création d'une vente 30	
3. Réception d'un achat fournisseur 31	
4. Rupture de stock et demande client 31	
X. Schéma de la base de données	31

1. Relations principales 32	
2. Index automatiques 32	
XI. Build, exécution et déploiement	32
1. Backend 33	
2. Frontend web 33	
XII. Couverture des fonctionnalités vs spécification	33
XIII. Perspectives d'évolution	34
1. Extension multi-magasins 34	
2. Application mobile pour la gestion de stock 35	
3. Tableau de bord analytique avancé 35	
4. Notifications automatiques 35	
5. Vitrine client et prise de commande en ligne 35	
6. Synthèse des priorités d'évolution 36	
XIV. Glossaire technique	36
XV. Annexes, bilan et conclusion générale	38
1. Exemple de gestion d'erreur 38	
2. Améliorations prévues 38	
3. Bilan personnel et conclusion générale 39	

I. Introduction et présentation du projet

BESTMARK est une application web développée dans le cadre d'un projet tutoré, destinée à la gestion globale d'un magasin de vente au détail. De nombreuses petites entreprises rencontrent des difficultés dans la gestion de leurs produits, commandes et clients : les processus manuels entraînent des erreurs et une perte de temps importante.

L'objectif du projet est de développer une application web permettant de gérer les produits, les commandes et les clients d'un magasin en ligne, de suivre les ventes et de générer des statistiques pour améliorer la gestion du commerce.

Le système couvre l'ensemble du cycle d'exploitation d'un point de vente : catalogue produits et catégories, fournisseurs et achats, ventes et facturation (avec gestion de la livraison et des paiements partiels), demandes clients (liste d'attente sur rupture de stock), gestion des employés et de leur paie, dépenses, mouvements de stock (traçabilité des entrées/sorties) et un registre financier centralisé (transactions).

Le système se décompose en deux applications complémentaires :

- Une API REST Node.js + Express adossée à une base MongoDB, qui centralise les données et expose l'ensemble des services métiers (produits, ventes, achats, stock, employés, finances...).
- Une application web React + Vite, utilisée par le personnel du magasin (administrateurs, employés, gestionnaires de stock) pour la gestion quotidienne de l'activité et la consultation des indicateurs.

1. Contexte et enjeux

De nombreux petits commerces de proximité continuent aujourd'hui de gérer leur activité avec des outils peu adaptés : cahiers papier, fichiers tableur isolés les uns des autres, ou logiciels de caisse basiques qui ne couvrent ni le suivi des fournisseurs, ni la gestion des employés, ni le suivi financier consolidé. Cette fragmentation a plusieurs conséquences concrètes : impossibilité de savoir rapidement quels produits sont en rupture, difficulté à recouper les ventes avec les dépenses pour connaître la marge réelle, absence de traçabilité en cas d'erreur de stock, et perte de temps considérable pour produire des statistiques même simples.

L'enjeu du projet BESTMARK est donc de démontrer, sur un cas concret, qu'une application web unique peut couvrir l'ensemble de ces besoins pour un commerce de taille modeste, sans nécessiter l'achat de plusieurs logiciels spécialisés ni de compétences techniques particulières pour l'utiliser au quotidien. Le choix d'une architecture web (accessible depuis un navigateur, sans installation) répond également à un enjeu pratique : le personnel doit pouvoir utiliser l'outil aussi bien depuis un poste fixe en caisse que depuis un ordinateur portable en réserve.

2. Objectifs de la mission

La mission confiée, telle que formalisée dans la fiche descriptive de mission professionnelle, poursuivait plusieurs objectifs concrets :

- concevoir un modèle de données couvrant l'ensemble du cycle d'exploitation d'un magasin (produits, fournisseurs, achats, ventes, stock, livraisons, clients, employés, finances) ;
- développer une API REST sécurisée exposant ces données aux applications clientes ;
- développer une interface web d'administration permettant au personnel de piloter l'activité quotidienne sans connaissance technique ;
- mettre en place un suivi de stock fiable et traçable, condition indispensable à la confiance des utilisateurs dans l'outil ;
- fournir une vision financière consolidée (ventes, achats, dépenses) pour faciliter le pilotage de l'activité.

Ces objectifs ont servi de fil conducteur tout au long des trois mois de développement, et ont été déclinés en checklists opérationnelles par module, comme détaillé dans le chapitre suivant consacré à la méthodologie de travail.

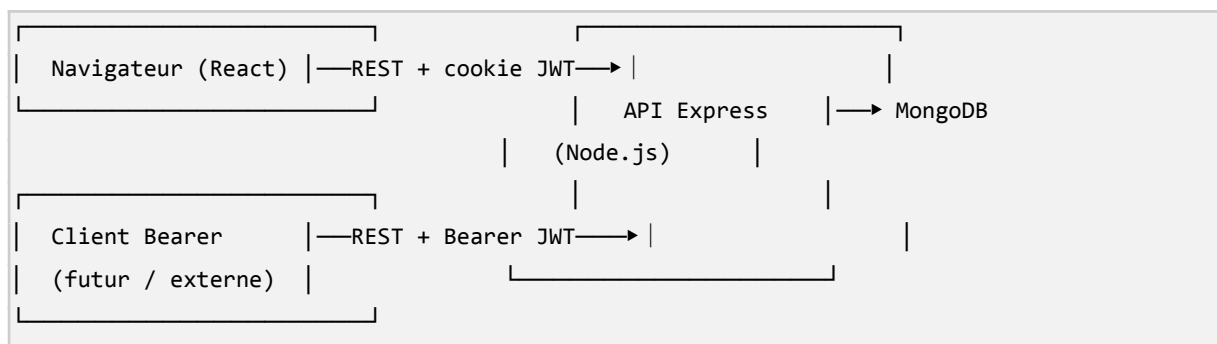
3. Pile technologique

Couche	Technologie	Version
Runtime backend	Node.js (modules CommonJS/ES mixtes)	—
Framework backend	Express	^5.2.1
Base de données / ODM	MongoDB / Mongoose	^9.5.0
Authentification	jsonwebtoken (JWT) + bcrypt	^9.0.3 / ^6.0.0

Couche	Technologie	Version
Sécurité HTTP	helmet, hpp, sanitation NoSQL maison	^8.1.0 / ^0.2.3
Limitation de débit	express-rate-limit (défini, désactivé)	^8.5.2
Upload / traitement images	multer + sharp	^2.2.0 / ^0.35.3
CORS / Cookies	cors / cookie-parser	^2.8.6 / ^1.4.7
Framework frontend	React	^19.2.5
Outil de build	Vite	^8.0.10
Routage frontend	react-router-dom	^7.14.2
Gestion des données serveur	@tanstack/react-query	^5.100.6
Tables de données	@tanstack/react-table	^8.21.3
UI Kit	MUI (Material UI) + Radix/shadcn + Tailwind	^9.0.0 / ^1.4.3 / ^4.2.4
Formulaires	react-hook-form	^7.74.0
Notifications	react-hot-toast	^2.6.0
Client HTTP (web)	axios	^1.15.2

4. Architecture de haut niveau

L'application web (navigateur) consomme l'API REST exclusivement via un jeton JWT transporté dans un cookie httpOnly (cookie « jwt »). Contrairement à un schéma double web/mobile, BESTMARK n'expose ici qu'un seul client (le frontend React) ; l'API accepte cependant aussi un en-tête Authorization: Bearer, ce qui laisse la porte ouverte à un futur client mobile ou à des intégrations externes.



Toutes les routes de ressources (à l'exception de la connexion) sont protégées par le middleware Protect, appliqué globalement dans Index.js avant chaque routeur métier.

5. Périmètre et limites du projet

Le périmètre de la mission a volontairement été circonscrit à une application de gestion mono-magasin, sans intégration avec un terminal de paiement physique ni avec un système de caisse existant. Cette délimitation, arrêtée dès la phase de cadrage, a permis de concentrer les trois mois de développement sur la couverture fonctionnelle en profondeur (neuf modules métier interconnectés) plutôt que sur l'intégration en largeur avec des systèmes tiers, jugée hors de portée pour une mission de cette durée.

De la même façon, l'application cible un usage interne par le personnel du magasin (administrateurs, employés, gestionnaires de stock) et ne comporte pas de vitrine e-commerce publique destinée aux clients finaux : les demandes clients (module 7 du chapitre III) sont saisies par le personnel, pas par le client lui-même via un compte en ligne. Cette limite de périmètre, assumée dès la fiche de mission, pourra constituer une piste d'évolution future, développée au chapitre XIII.

6. Public cible et utilisateurs de l'application

Trois profils d'utilisateurs ont guidé les choix de conception, à travers le champ role du modèle employé : l'administrateur, qui dispose d'une vision complète sur l'ensemble des modules (finances comprises) et gère les comptes du personnel ; l'employé, qui utilise principalement les modules Ventes, Livraisons et Demandes clients au quotidien ; et le gestionnaire de stock, davantage concerné par les modules Produits, Achats et Mouvements de stock. Bien que la distinction fonctionnelle entre ces trois profils soit déjà présente dans le modèle de données, leur application effective sous forme de restrictions d'accès côté serveur reste, comme indiqué au chapitre IV, une évolution à finaliser.

Cette réflexion sur les profils utilisateurs a néanmoins influencé la conception des écrans frontend dès cette première version : la barre latérale de navigation (Sidebar) a été pensée pour pouvoir, à terme, masquer certains modules selon le rôle connecté, même si cette adaptation dynamique de l'interface n'est pas encore branchée sur le champ role en base.

II

II. Méthodologie de travail

Le développement de BESTMARK s'est déroulé sur une période d'environ trois mois, en autonomie complète, dans le cadre du projet tutoré mené au sein de l'école. En l'absence d'une équipe ou d'un chef de projet externe, l'enjeu principal a été de structurer le travail de façon à couvrir méthodiquement

l'ensemble des besoins métiers identifiés dans la fiche de mission, sans perdre de vue la cohérence globale entre les différents modules (produits, stock, achats, ventes, livraisons, finances...).

1. Outil de suivi : Notion

La gestion du projet a été organisée dans Notion, sous la forme d'un tableau de bord découpé par domaine métier (onglets « Achats », « Mouvement de stock », « Produit », etc., un onglet par module fonctionnel de l'application). Chaque onglet contient une checklist décrivant, avant tout développement, les règles métier que le module doit respecter une fois terminé.

Par exemple, pour le module Achats, la checklist Notion établie en amont précisait :

- ajouter un modèle de données pour l'achat ;
- lors d'un achat, sélectionner le produit concerné et incrémenter sa quantité en stock ;
- enregistrer le montant de l'opération dans le suivi financier, avec son type (achat) ;
- ajouter le mouvement de stock correspondant, pour tracer l'entrée ;
- ajouter une livraison si le produit acheté en nécessite une (cas des commandes fournisseurs livrées directement en magasin).

Cette même logique de checklist a ensuite été déclinée pour chacun des autres modules (Ventes, Produits, Fournisseurs, Employés, Dépenses...), ce qui a permis de garder une trace écrite des règles métier à respecter et de vérifier, module par module, qu'aucune règle n'était oubliée avant de passer à la suite.

2. Organisation du développement

Le travail n'a pas suivi un formalisme de type Scrum avec des sprints définis à l'avance, mais une progression itérative module par module : pour chaque domaine métier, le modèle de données (Mongoose) était conçu en premier, suivi du contrôleur et des routes REST associées côté backend, puis de l'écran et des composants React correspondants côté frontend. Un module n'était considéré comme terminé que lorsque l'ensemble de sa checklist Notion était cochée.

Cette organisation a permis de livrer des briques fonctionnelles complètes et testables individuellement (par exemple, le module Produits était pleinement opérationnel — CRUD et upload d'image compris — avant que le module Ventes, qui en dépend, ne soit commencé), réduisant le risque de devoir revenir en arrière sur des fondations mal posées.

3. Planning prévisionnel

Le projet s'est déroulé sur trois mois, découpés en quatre grandes phases. Ce découpage n'a pas été figé au jour près, mais a servi de repère pour vérifier régulièrement que l'avancement restait cohérent avec le temps disponible.

Période	Phase	Contenu principal
Mois 1 (semaines 1-4)	Fondations	Modélisation de la base de données complète, mise en place de l'authentification (JWT, bcrypt), modules Produits et Catégories (backend + frontend)
Mois 1-2 (semaines 4-7)	Cœur métier — approvisionnement	Modules Fournisseurs, Achats et Mouvements de stock ; premiers écrans d'administration
Mois 2 (semaines 7-10)	Cœur métier — vente	Module Ventes (le plus complexe fonctionnellement), Livraisons, Demandes clients
Mois 2-3 (semaines 10-12)	Ressources humaines et finances	Modules Employés, Bulletins de paie, Dépenses, Transactions centralisées
Mois 3 (semaines 12-13)	Sécurisation et finalisation	Durcissement de la sécurité de l'API, gestion centralisée des erreurs, tests manuels, rédaction de la documentation

4. Environnement de travail et outils

Au-delà de Notion pour le suivi fonctionnel, plusieurs outils ont accompagné le développement au quotidien :

- Visual Studio Code comme éditeur principal, avec les extensions ESLint et Prettier pour maintenir un style de code cohérent sur l'ensemble du projet.
- Un client HTTP (type Postman/Insomnia) pour tester manuellement chaque endpoint de l'API au fur et à mesure de son développement, avant même la mise en place de l'écran frontend correspondant.
- MongoDB Compass pour inspecter directement le contenu de la base de données, vérifier la structure des documents créés et diagnostiquer d'éventuels problèmes de relations entre collections.

- Git pour le versionnement du code, avec le dépôt hébergé sur GitHub pour conserver un historique des évolutions du projet.
- Le navigateur (outils de développement, onglet Réseau) pour observer les échanges entre le frontend React et l'API, notamment lors de la mise au point de l'authentification par cookie httpOnly.

III**III. Description de la mission — développement par module métier**

Cette partie détaille, module par module, le travail réalisé dans le cadre de la mission : le besoin métier à l'origine du module, les choix de conception retenus, et le fonctionnement du module une fois terminé, illustré par le déroulement concret d'une opération type.

1. Gestion des produits et catégories*Besoin métier*

Le point de départ du projet a été la mise en place du catalogue produits, socle sur lequel reposent ensuite les achats, les ventes et le suivi de stock. Sans un référentiel produit fiable, aucun des autres modules ne pouvait être développé de façon cohérente : c'est donc naturellement le premier module qui a été construit, avant même le module Ventes pourtant plus visible pour un utilisateur final.

Conception

Chaque produit regroupe un nom, une description, un prix d'achat et un prix de vente, une image, une catégorie de rattachement et une quantité en stock. Un seuil d'alerte de rupture (minStockAlert, fixé à 5 par défaut) a été ajouté pour permettre, par la suite, de détecter automatiquement les produits à réapprovisionner. Un produit peut optionnellement être rattaché à un fournisseur (hassupplier / supplier) : cette relation conditionnelle a demandé de mettre en place une validation Mongoose dynamique, où le champ supplier n'est obligatoire que si hassupplier vaut true, afin de ne pas imposer de fournisseur aux produits qui n'en ont pas besoin (cas de produits fabriqués en interne, par exemple).

La gestion des catégories, plus simple, associe un nom et un slug à chaque famille de produits, pour faciliter le filtrage côté interface et préparer une éventuelle navigation par URL propre (ex. /categories/electromenager).

Fonctionnement

- L'administrateur accède à l'écran ProductsPage, qui affiche la liste paginée du catalogue avec un filtre (Filter.jsx) par catégorie, statut de stock ou recherche par nom.
- À la création d'un produit, un formulaire à deux volets (AddMemberModal-like) collecte les informations générales puis l'image, envoyée via multer et redimensionnée côté serveur avec sharp avant stockage.
- Le compteur Number_of_sales, initialisé à zéro, est incrémenté automatiquement à chaque vente enregistrée, ce qui alimente le module de statistiques (ProductsStatic.jsx) sans calcul à la volée.
- Lorsque la quantité d'un produit descend sous son minStockAlert, le produit devient éligible à une demande client en liste d'attente (module 7) et à un réapprovisionnement via le module Achats (module 3).

Résultat

Le module Produits et Catégories est aujourd'hui pleinement fonctionnel, upload et redimensionnement d'image compris, et constitue la base stable sur laquelle tous les autres modules métiers s'appuient.

2. Gestion des fournisseurs

Besoin métier

Le module Fournisseurs a été conçu pour centraliser les partenaires auprès desquels le magasin se réapprovisionne, condition préalable à la mise en place du module Achats qui référence systématiquement un fournisseur.

Conception

Un fournisseur regroupe un nom, une société, des coordonnées (email, téléphone), une adresse, et surtout une relation productTypes vers plusieurs catégories de produits. Cette relation permet, à terme, de filtrer rapidement quel fournisseur contacter pour réapprovisionner une catégorie de produits donnée, plutôt que de devoir parcourir la liste complète des fournisseurs à chaque nouvelle commande.

Fonctionnement

- Sur SuppliersPage, l'équipe consulte la liste des fournisseurs et peut en ajouter un nouveau via un formulaire simple (CRUD standard).
- Lors de la création d'un achat (module 3), le fournisseur est sélectionné dans une liste déroulante alimentée par ce module, ce qui garantit la cohérence référentielle entre les deux modules.

Résultat

Le module Fournisseurs est opérationnel et pleinement intégré au module Achats ; son enrichissement futur le plus utile serait un historique consolidé des commandes passées par fournisseur, directement accessible depuis sa fiche.

3. Gestion des achats (réapprovisionnement)

Besoin métier

Une fois le catalogue et les fournisseurs en place, il devenait possible de modéliser le processus de réapprovisionnement : comment le stock d'un produit est reconstitué, et comment cette dépense est répercutée sur la comptabilité du magasin.

Conception

Une commande d'achat (Purchase) regroupe une liste d'articles (produit, quantité, prix d'achat), un montant total, un montant déjà payé et une dette éventuelle, avec un statut de paiement (payé, partiel, impayé) qui reflète la réalité d'une petite structure où le fournisseur n'est pas toujours réglé intégralement à la livraison.

Fonctionnement

- L'administrateur sélectionne un fournisseur, ajoute un ou plusieurs produits avec leur quantité et leur prix d'achat, puis valide la commande.
- Conformément à la checklist Notion établie pour ce module, la validation déclenche trois effets : l'incrémentation du stock des produits reçus,
- la création d'un mouvement de stock de type « purchase » pour assurer la traçabilité de l'entrée,
- et l'enregistrement d'une transaction financière de type « purchase » (direction « out ») dans le registre centralisé (module 9).
- Si le montant payé est inférieur au montant total, la dette (debts) est calculée et conservée pour suivi, permettant de savoir à tout moment ce qui reste dû à chaque fournisseur.

Résultat

Le module Achats fonctionne de bout en bout, avec un enchaînement fiable entre la commande, la mise à jour du stock, le mouvement de stock et la transaction financière — les quatre effets attendus de la checklist Notion initiale sont bien couverts.

4. Gestion du stock (mouvements et traçabilité)

Besoin métier

Une simple quantité en stock sur chaque produit aurait suffi pour un prototype, mais ne permet pas de répondre à une question essentielle pour un commerçant : pourquoi le stock d'un produit a-t-il changé ? Ce module répond à ce besoin de traçabilité.

Conception

Plutôt que de se contenter de modifier directement la quantité en stock d'un produit, il a été décidé de journaliser chaque variation dans une collection dédiée, StockMovement : produit concerné, type de mouvement (achat, vente, retour, ajustement manuel), quantité avant et après l'opération (quantityBefore / quantityAfter), et référence vers le document d'origine (achat ou vente). Cette approche, plus lourde à mettre en place qu'une simple mise à jour de quantité, offre un historique complet et auditable, utile pour investiguer un écart d'inventaire constaté lors d'un contrôle physique du stock.

Fonctionnement

- Chaque fois qu'une vente, un achat ou un ajustement manuel modifie la quantité d'un produit, un StockMovement est créé automatiquement avec l'état avant/après.
- L'écran StockPage permet de consulter cet historique produit par produit, et d'enregistrer un ajustement manuel (par exemple après un inventaire physique révélant un écart).
- Ce journal constitue la source de vérité en cas de litige sur une quantité, plutôt que de devoir faire confiance uniquement à la valeur courante du champ quantity du produit.

Résultat

La traçabilité complète des mouvements de stock est opérationnelle ; seul un index composé optimisant les requêtes de filtrage par produit et par date (identifié au chapitre X) reste à ajouter pour fiabiliser les performances à plus grande échelle.

5. Gestion des ventes et de la facturation

Besoin métier

Le module Ventes est le cœur de l'activité quotidienne du magasin : c'est celui que le personnel utilise le plus souvent, et celui dont dépendent directement le chiffre d'affaires et la satisfaction client.

Conception

Une vente (Sale) regroupe les articles vendus, le calcul du sous-total, d'une remise éventuelle et du montant total, ainsi que le mode de paiement (espèces, carte, virement) et le statut de paiement (payé, partiel, remboursé, impayé). La possibilité de vente à crédit a été intégrée dès la conception du modèle, via les champs `paidAmount` (montant réellement encaissé) et `remainAmount` (reste dû calculé automatiquement) — une situation fréquente pour un commerce de proximité qui fait crédit à des clients réguliers.

Fonctionnement

- Sur `CheckinPage/SalesPage`, le vendeur recherche et sélectionne les produits, chacun avec sa quantité (le formulaire a été découpé en sous-composants : `ProductPart` pour la sélection des articles, `CustomerPart` pour les informations client en cas de livraison, `PaymentPart` pour le règlement, et `FactureSell` pour la facture).
- Le sous-total, la remise et le total sont calculés automatiquement au fil de la saisie.
- Selon le montant réellement encaissé (`paidAmount`), le statut de paiement est déterminé, et le reste dû (`remainAmount`) est enregistré si la vente n'est pas intégralement réglée.
- Si la vente nécessite une livraison, un document `Delivery` est créé et lié à la vente (module 6).
- Le stock des produits vendus est décrémenté, un mouvement de stock de type « sale » est journalisé, et une transaction financière de type « sale » (direction « in ») est enregistrée.

Résultat

Le module Ventes, malgré sa complexité (paiement partiel, livraison optionnelle, facturation), fonctionne de bout en bout et a été le module le plus testé manuellement, du fait de son usage quotidien intensif attendu par le personnel du magasin.

6. Gestion des livraisons

Besoin métier

Certains clients souhaitent être livrés plutôt que d'emporter leurs achats sur place ; il fallait donc prévoir un suivi minimal du processus de livraison, rattaché à la vente qui l'a déclenché.

Conception

Lorsqu'une vente nécessite une livraison (champ `requiresDelivery`), un document `Delivery` lui est associé, avec une adresse de livraison, des frais de livraison et un statut qui évolue au fil du processus : en attente, en préparation, en route, arrivée, ou échouée.

Fonctionnement

- À la création d'une vente avec livraison, l'adresse et les frais de livraison sont saisis dans CustomerPart, puis un document Delivery est créé en parallèle du document Sale.
- Le statut est ensuite mis à jour manuellement par l'équipe au fur et à mesure de l'avancement de la livraison.
- Ce module reste volontairement simple à ce stade du projet (pas encore de suivi temps réel ni de notification automatique), mais pose les bases nécessaires — modèle de données, statuts, association à la vente — pour une future intégration d'un suivi live si le projet devait être poursuivi au-delà de la mission.

Résultat

Le module Livraisons couvre le besoin minimal identifié (suivi de statut rattaché à une vente) ; son évolution vers un suivi temps réel ou une notification automatique du client est développée au chapitre XIII, consacré aux perspectives d'évolution du projet.

7. Gestion des demandes clients (liste d'attente)

Besoin métier

Ce module n'était pas demandé explicitement dans la spécification initiale de la fiche de mission, mais a été ajouté après avoir identifié, pendant la conception du module Produits, un besoin métier concret : que faire lorsqu'un client souhaite un produit actuellement en rupture de stock ?

Conception

Une demande client (CustomerRequest) regroupe le nom et le téléphone du client, le produit et la quantité souhaités, avec un statut qui évolue de « en attente » à « notifié » (une fois le produit réapprovisionné, avec horodatage notifiedAt) puis « honoré » (une fois la vente réalisée), ou « annulé ».

Fonctionnement

- Lorsqu'un client demande un produit en rupture, l'équipe enregistre sa demande sur CustomersPage plutôt que de le renvoyer sans solution.
- Lorsque le produit est réapprovisionné (module 3), l'équipe peut consulter les demandes en attente pour ce produit et les marquer comme « notifiées ».

- Une fois la vente effectivement réalisée avec ce client, la demande est marquée « honorée », ce qui permet de mesurer combien de ruptures de stock ont finalement abouti à une vente malgré tout.

Résultat

Ce module, ajouté en cours de projet suite à un besoin identifié plutôt qu'imposé dès le départ, illustre la souplesse de l'organisation par checklists Notion : il a pu être inséré sans remettre en cause les modules déjà terminés.

8. Gestion des employés et de la paie

Besoin métier

Au-delà de l'authentification, un commerce a besoin de suivre son personnel : qui travaille, à quel poste, pour quel salaire, et où en sont les versements de paie.

Conception

Le module Employés porte à la fois la gestion du personnel (nom, coordonnées, poste occupé via une référence vers EmployeeType, salaire, statut actif/inactif) et l'authentification de l'application, chaque employé disposant d'un compte de connexion. Un sous-module complémentaire, FactureEmploi, permet d'enregistrer les bulletins de paie mensuels (salaire, avance, reste à verser, total déjà versé), afin de garder un historique des versements effectués à chaque employé plutôt qu'une simple ligne de salaire brut.

Fonctionnement

- Un administrateur crée un compte employé avec son poste (mission) et son salaire de référence, via EmployeesPage.
- Chaque mois, un bulletin FactureEmploi peut être généré pour cet employé, avec le montant versé et le reste éventuel si une avance a déjà été donnée en cours de mois.
- Un compte employé peut être désactivé (plutôt que supprimé) en cas de départ, ce qui préserve l'historique des ventes et actions qu'il a effectuées tout en lui coupant l'accès à l'application.

Résultat

Le module Employés et l'authentification associée sont pleinement opérationnels ; la couverture du sous-module FactureEmploi côté interface reste à finaliser, comme indiqué dans le tableau de couverture du chapitre XII.

9. Suivi financier centralisé (dépenses et transactions)

Besoin métier

Une fois les modules Ventes, Achats et Employés en place, il manquait une vue d'ensemble de la santé financière du magasin : combien rentre, combien sort, et pour quelle raison — sans avoir à recouper manuellement plusieurs collections.

Conception

Un registre Transaction centralise tous les mouvements d'argent, qu'ils proviennent d'une vente (entrée), d'un achat ou d'une dépense (sortie). Chaque transaction référence son document d'origine de façon polymorphe (referenceModel + referenceId), ce qui permet de retrouver facilement le détail d'une opération financière à partir du registre, sans dupliquer l'information déjà présente dans les modules Ventes, Achats ou Dépenses. Un module Dépenses (Expense / ExpenseType) complète ce suivi pour les charges qui ne sont ni des achats de marchandises ni des salaires (loyer, électricité, fournitures diverses).

Fonctionnement

- Chaque vente, achat ou dépense validée crée automatiquement une entrée correspondante dans le registre Transaction, avec son sens (entrée/sortie) et son montant.
- L'écran TransactionsPage offre une vue consolidée de ces flux, permettant à l'administrateur de suivre la trésorerie du magasin sans avoir à ouvrir séparément les modules Ventes, Achats et Dépenses.
- Ce module a été conçu en dernier, une fois les modules producteurs d'opérations financières (Ventes, Achats, Dépenses) déjà en place, afin de s'assurer que le registre centralisé reflète fidèlement des besoins réels plutôt que des hypothèses.

Résultat

Ce dernier module boucle la couverture fonctionnelle de la mission : chaque flux d'argent généré par les huit autres modules trouve sa place dans un registre unique, ce qui répond directement à l'objectif de vision financière consolidée fixé au chapitre I.

IV

IV. Difficultés rencontrées et compétences acquises

Le projet n'a pas connu de blocage majeur ayant remis en cause son avancement global : la progression module par module, associée aux checklists Notion, a permis d'anticiper la plupart des besoins avant de coder. En revanche, deux aspects transverses ont représenté un véritable travail d'apprentissage, en particulier sur la partie backend.

1. Sécurisation de l'API

Avant ce projet, la sécurisation d'une API REST se limitait, dans mon expérience, à la simple vérification d'un jeton JWT. BESTMARK a été l'occasion de comprendre et de mettre en œuvre une chaîne de sécurité plus complète : hachage des mots de passe avec bcrypt, en-têtes de sécurité HTTP via helmet, protection contre la pollution de paramètres avec hpp, et une sanitation NoSQL maison pour neutraliser les tentatives d'injection d'opérateurs MongoDB dans le corps, les paramètres de requête et l'URL. Comprendre précisément ce que chacune de ces couches protège — et pourquoi une seule d'entre elles ne suffit pas — a demandé des recherches et plusieurs itérations, notamment pour vérifier que la sanitation ne cassait pas les requêtes légitimes.

2. Gestion centralisée des erreurs

La deuxième difficulté a été de sortir d'une gestion d'erreurs dispersée, avec un bloc try/catch répété dans chaque fonction contrôleur, pour aller vers un mécanisme centralisé. La mise en place du wrapper catchAsync, d'une classe d'erreur applicative dédiée (APPErrors) et d'un gestionnaire d'erreur global unique a demandé de repenser la façon dont les erreurs remontent dans l'application Express, en particulier pour bien comprendre le rôle du paramètre next(err) et la manière dont Express distingue un middleware normal d'un middleware de gestion d'erreurs (à quatre paramètres). Ce travail a aussi mis en évidence une limite encore présente dans la version actuelle du projet : le gestionnaire d'erreur ne différencie pas encore systématiquement tous les codes HTTP renvoyés, ce qui reste une piste d'amélioration identifiée pour la suite.

3. Modélisation des relations de données complexes

Le domaine métier de BESTMARK comporte seize collections MongoDB interconnectées, avec des relations de nature très différente : relations simples (un produit appartient à une catégorie), relations conditionnelles (un produit n'a de fournisseur que s'il en dépend réellement), et références polymorphes

(une transaction ou un mouvement de stock peut référencer indifféremment une vente, un achat ou une dépense, selon la valeur d'un champ `referenceModel`). Cette dernière catégorie a été la plus délicate à appréhender : Mongoose ne permettant pas nativement le `populate()` d'une référence polymorphe aussi simplement que pour une relation classique, il a fallu comprendre comment structurer les champs (`referenceModel` comme chaîne de caractères, `referenceId` comme `ObjectId` générique) pour pouvoir retrouver le bon document d'origine à la lecture, tout en gardant un modèle de données lisible.

4. Organisation du code par domaine métier

Avec neuf modules métier distincts à développer en autonomie, le risque de finir avec un code désorganisé était réel. Le choix a été fait, dès le départ, d'organiser aussi bien le backend que le frontend par domaine fonctionnel plutôt que par type technique : chaque dossier backend (`Products/`, `Purchases/`, `sales/...`) regroupe modèle, contrôleur et routeur d'un même module, et chaque dossier frontend (`Component/Products/`, `Component/Sales/...`) regroupe la page, le tableau de données et les actions CRUD correspondantes. Ce choix, qui peut sembler évident a posteriori, a demandé une réflexion en amont sur la structure du dépôt avant même d'écrire la première ligne de code métier, afin d'éviter d'avoir à réorganiser le projet en cours de route une fois plusieurs modules déjà développés.

5. Compétences acquises

Authentification par JWT stocké en cookie `httpOnly` : ce projet a été l'occasion de comprendre non seulement comment émettre et vérifier un jeton, mais aussi comment le transporter de façon sécurisée selon le type de client (cookie `httpOnly` pour le navigateur, en-tête `Authorization: Bearer` pour un client externe potentiel), et comment invalider un jeton existant lors d'un changement de mot de passe.

Hachage sécurisé des mots de passe avec `bcrypt`, et compréhension des bonnes pratiques associées : ne jamais stocker ni transmettre un mot de passe en clair, utiliser un nombre de rounds de salage suffisant, et ne re-hasher le mot de passe que lorsqu'il a effectivement été modifié (garde `isModified` côté Mongoose).

Durcissement HTTP d'une API Express : mise en œuvre concrète de `helmet` pour les en-têtes de sécurité, de `hpp` contre la pollution de paramètres, d'une limite de taille sur les requêtes JSON, et désactivation de l'en-tête `x-powered-by` pour ne pas exposer inutilement la technologie serveur utilisée.

Prévention des injections NoSQL par une fonction de sanitation récursive développée spécifiquement pour ce projet, capable de neutraliser toute clé d'opérateur MongoDB (préfixée par `$`) dans le corps, les paramètres de requête et l'URL, sans dépendre uniquement d'un paquet tiers.

Conception d'un mécanisme centralisé de gestion des erreurs asynchrones : wrapper `catchAsync` pour éliminer la répétition de blocs `try/catch`, classe d'erreur applicative dédiée (`APPError`) portant un code HTTP et un statut, et gestionnaire d'erreur global capable de différencier plusieurs types d'erreurs (JWT invalide, erreur de validation Mongoose, erreur Multer sur l'upload d'image).

Modélisation de données relationnelles avec Mongoose sur un domaine métier riche : références `ObjectId` simples, `populate()` pour peupler les documents liés, validations conditionnelles (un champ requis seulement si une autre condition est vraie), et références polymorphes pour le registre financier centralisé — sur un total de seize collections interconnectées.

Conception d'une API REST cohérente sur un périmètre fonctionnel large : neuf modules métier (produits, fournisseurs, achats, stock, ventes, livraisons, demandes clients, employés, finances) exposés selon un schéma d'endpoints homogène, ce qui a renforcé ma capacité à maintenir une convention de nommage et de structure stable même lorsque le nombre de ressources augmente significativement.

V

V. Tests et validation

En l'absence de suite de tests automatisés à ce stade du projet (identifiée comme axe d'amélioration au chapitre XII), la validation du bon fonctionnement de chaque module a reposé sur des tests manuels systématiques, menés au fur et à mesure du développement plutôt qu'en une seule phase finale.

1. Tests de l'API au fur et à mesure du développement

Chaque endpoint REST a été testé manuellement avec un client HTTP (Postman/Insomnia) dès sa création, avant même le développement de l'écran frontend correspondant. Cette approche a permis de valider indépendamment la logique métier côté backend (calculs, validations Mongoose, effets de bord comme la création automatique d'un mouvement de stock) sans dépendre de l'interface, et de détecter plus tôt les erreurs de validation ou les cas limites mal gérés (par exemple, une quantité négative envoyée dans le corps d'une requête d'achat).

2. Scénarios de bout en bout testés manuellement

Une fois chaque module intégré au frontend, plusieurs scénarios métier complets ont été rejoués manuellement pour vérifier la cohérence de bout en bout entre les différents modules :

- Connexion d'un employé actif, puis tentative de connexion avec un compte désactivé (isActive à false), pour vérifier le message d'erreur et le refus d'accès.
- Création d'une vente avec paiement partiel : vérification que le montant restant dû (remainAmount) est correctement calculé, affiché, et que le statut de paiement passe bien à « partiel ».
- Vente d'un produit jusqu'à épuisement du stock, puis création d'une demande client pour ce même produit, pour vérifier l'enchaînement entre les modules Ventes, Produits et Demandes clients.
- Réception d'un achat fournisseur, puis vérification dans l'écran Stock que le mouvement d'entrée correspondant apparaît bien avec la bonne quantité avant/après.
- Création d'une vente avec livraison, suivie de la mise à jour manuelle du statut de livraison jusqu'à « arrivée », pour vérifier la cohérence entre les modules Ventes et Livraisons.
- Tentative d'accès à une route protégée sans jeton, puis avec un jeton expiré ou invalide, pour vérifier que le middleware Protect renvoie bien une erreur 401 dans chaque cas.

3. Vérification de la sécurité de base

Au-delà des scénarios fonctionnels, quelques vérifications de sécurité ont été menées manuellement : tentative d'injection d'un opérateur MongoDB (par exemple { "\$gt": "" }) dans un champ de recherche pour vérifier l'efficacité de la sanitation NoSQL maison, envoi d'un corps de requête dépassant la limite de 10 ko pour vérifier le rejet attendu, et upload d'une image dépassant la taille maximale autorisée pour vérifier le message d'erreur dédié renvoyé par le gestionnaire d'erreur global (LIMIT_FILE_SIZE).

Ces vérifications manuelles, bien que non automatisées, ont permis de valider empiriquement que les mécanismes de sécurité mis en place au chapitre IV fonctionnaient comme prévu, avant la rédaction de la documentation technique détaillée dans les chapitres suivants.

4. Bilan des tests et limites de l'approche manuelle

L'ensemble des scénarios listés ci-dessus a été rejoué avec succès sur la version actuelle du projet, ce qui donne un niveau de confiance raisonnable sur le fonctionnement des parcours métier principaux. Cette

approche manuelle a toutefois des limites assumées : elle ne garantit pas une couverture exhaustive de tous les cas limites possibles (par exemple, les comportements en cas de perte de connexion réseau en cours de vente n'ont pas été systématiquement explorés), et elle ne protège pas contre les régressions qui pourraient être introduites par une modification future du code, faute de suite de tests automatisés rejouable à chaque changement.

C'est précisément cette limite qui motive l'écriture de tests automatisés comme priorité haute dans la feuille de route présentée au chapitre XIII (section 6, synthèse des priorités d'évolution) : transformer les scénarios manuels décrits dans ce chapitre en tests Jest et Vitest permettrait de les rejouer systématiquement, sans effort humain répété, à chaque évolution du code.

VI

VI. Structure du dépôt

Le projet est organisé en deux dossiers de premier niveau : Back-end (API Express + Mongoose) et Front-end (application React + Vite). Chaque ressource métier du backend suit une organisation modulaire par domaine (modèle, contrôleur, routeur regroupés dans un même dossier).

```
BESTMARK/
├── Back-end/
│   ├── Customers/           # Demandes clients (liste d'attente produit)
│   │   ├── CustomerRequest.js
│   │   ├── Controller.js
│   │   └── Router.js
│   ├── Delivery/           # Livraisons liées aux ventes
│   ├── Employes/
│   │   ├── Emplpye/        # Employés + authentification
│   │   │   ├── Employee.js
│   │   │   ├── AuthEmployee.js
│   │   │   ├── Controller.js
│   │   │   └── Router.js
│   │   ├── typeemplois/    # Types de poste (mission)
│   │   └── FactureEmployer/ # Bulletins de paie (FactureEmploi)
│   ├── Midelwars/
│   │   ├── ErrorHandler.js # Gestionnaire d'erreur global
│   │   └── UploadImage.js   # Upload + optimisation image (multer/sharp)
│   ├── Products/
│   │   ├── Product/        # Catalogue produits
│   │   └── Productcategories/ # Catégories de produits
│   └── Purchases/          # Achats fournisseurs
```

```

|   └─ Suppliers/           # Fournisseurs
|   └─ Transactions/        # Registre financier centralisé
|   └─ Users/              # Comptes utilisateurs (données)
|   └─ expenses/
|       └─ expense/         # Dépenses
|       └─ ExpensesType/    # Types de dépense
|   └─ sales/              # Ventes / facturation
|   └─ stockMovements/      # Journal des mouvements de stock
|   └─ utils/
|       └─ CatchFunction.js  # Wrapper asyncHandler + transactions Mongo
|       └─ ErrorHandler.js   # Classe APPError
|   └─ Index.js             # Point d'entrée
|   └─ package.json
└─ Front-end/
    └─ src/
        └─ Component/       # Un sous-dossier par domaine métier
            └─ Products/ Suppliers/ Employees/ Purchases/
            └─ Stock/ Sales/ Delivery/ Customers/ Transactions/
            └─ Settings/ (types, catégories, profil)
            └─ Home/ (Sidebar, NavBar, page d'accueil)
            └─ Login/
            └─ ErrorPage/
        └─ Services/         # Wrappers Axios par ressource
            └─ axiosInstance.js
        └─ Component/Data/contextApi.jsx # Contexte global React
        └─ AppRouter.jsx     # Déclaration des routes
        └─ App.jsx
        └─ main.jsx
    └─ vite.config.js
    └─ package.json

```

VII**VII. Backend — API REST****1. Cycle de vie du serveur**

Le fichier Index.js orchestre le démarrage de l'application. Les étapes suivantes s'enchaînent :

- Chargement des variables d'environnement via dotenv.config().

- Activation de helmet() (en-têtes de sécurité HTTP) dès le début de la chaîne de middlewares.
- Exposition du dossier /uploads en statique, avec un en-tête Cross-Origin-Resource-Policy: cross-origin pour permettre l'affichage des images produits/employés depuis le frontend.
- Connexion à MongoDB via mongoose.connect(process.env.DataBase).
- Configuration CORS restreinte à l'origine http://localhost:5173, avec credentials: true pour autoriser l'envoi du cookie JWT.
- Parsing JSON limité à 10 ko (express.json({ limit: '10kb' })) et lecture du cookie via cookieParser().
- Sanitation NoSQL maison : une fonction récursive supprime toute clé commençant par « \$ » dans req.body, req.query et req.params, afin de neutraliser les tentatives d'injection d'opérateurs MongoDB.
- Protection contre la pollution de paramètres HTTP via hpp().
- Désactivation de l'en-tête x-powered-by.
- Montage de l'ensemble des routeurs métiers, tous précédés du middleware Protect (sauf la route de connexion).
- Route catch-all (regex /.*/) qui délègue à handleUnhanledRoute pour générer une erreur 404 explicite.
- Gestionnaire d'erreur global qui uniformise les réponses JSON d'erreur.
- Démarrage du serveur sur le port défini par la variable d'environnement PORT.

Point notable : le limiteur de débit express-rate-limit est déjà codé (100 requêtes / heure) mais reste commenté dans Index.js — il n'est donc pas actif dans la version actuelle.

2. Table des endpoints REST

L'ensemble des ressources suit un schéma CRUD REST cohérent. Toutes les routes ci-dessous sont protégées par le middleware Protect (JWT), à l'exception de la connexion.

Ressource	Méthode	Chemin	Description
Auth	POST	/api/auth/login	Connexion employé (public)
Employés	GET	/api/employees/me	Profil de l'employé connecté
Employés	POST	/api/employees/	Créer un employé (upload photo)
Employés	GET	/api/employees/	Lister les employés

Ressource	Méthode	Chemin	Description
Employés	GET / PATCH / DELETE	/api/employees/:id	Détail / modifier / supprimer
Employés	PUT	/api/employees/password/:id	Changer le mot de passe
Employés	PUT	/api/employees/Desactiver/Account/:id	Activer / désactiver le compte
Employés	POST	/api/employees/logout	Déconnexion
Types d'employé	CRUD	/api/employee-types/	Types de poste (mission)
Produits	CRUD	/api/products/	Catalogue produits (upload image)
Catégories	CRUD	/api/categories/products/	Catégories de produits
Fournisseurs	CRUD	/api/suppliers/	Fournisseurs
Achats	CRUD	/api/purchases/	Commandes fournisseurs
Ventes	CRUD	/api/sales/	Ventes / facturation
Livraisons	CRUD	/api/delivery/	Livraisons liées aux ventes
Demandes clients	CRUD	/api/customers/	Liste d'attente produit
Mouvements stock	CRUD	/api/stock-movements/	Journal des entrées/sorties
Dépenses	CRUD	/api/expenses/	Dépenses
Types de dépense	CRUD	/api/expense-types/	Types de dépense
Transactions	CRUD + GET /reference/:ref	/api/transactions/	Registre financier centralisé
Utilisateurs	CRUD	/api/users/	Comptes utilisateurs (données)

3. Modèles de données Mongoose

Le modèle de données comporte seize collections, réparties en quatre grands ensembles fonctionnels.

3.1. Identité et personnel

Champ	Type	Contraintes
name	String	obligatoire
phone	Number	obligatoire, unique
role	String	enum [admin, employee, stock_manager]
password	String	obligatoire, haché bcrypt (10 rounds), min. 8 caractères
isActive	Boolean	défaut true

User (compte utilisateur) — modèle de données distinct du modèle Employee, dont le champ role servira de base à la future différenciation des droits par profil.

Champ	Type	Contraintes
name	String	obligatoire, unique
email	String	unique, validé (validator.isEmail)
phone	String	obligatoire
mission	ObjectId → EmployeeType	obligatoire
image	String	chemin de l'image uploadée
salary	Number	obligatoire
password	String	obligatoire, haché bcrypt (12 rounds), select: false
isActive	Boolean	défaut true
passwordchangeafter	Date	invalide les jetons émis avant un changement de mot de passe
AccountDesactivateDate	Date	horodatage de (dés)activation du compte

Employee (employé) — porteur de l'authentification de l'application.

3.2. Catalogue et fournisseurs

Champ	Type	Contraintes
name	String	obligatoire
description	String	obligatoire
barcode	String	optionnel
buyingPrice / sellingPrice	Number	obligatoires
image	String	obligatoire
category	ObjectId → Category	obligatoire
quantity	Number	défaut 0
hasupplier / supplier	Boolean / ObjectId → Supplier	supplier requis si hasupplier = true
minStockAlert	Number	défaut 5 (seuil d'alerte de rupture)
Number_of_sales	Number	défaut 0 (compteur de ventes)

Product (produit) — inclut un virtual productprofit (sellingPrice – buyingPrice).

- Category : name, slug
- Supplier : name, company, email, phone, address, productTypes (liste de catégories fournies)

3.3. Achats, ventes, livraison et clients

Modèle	Champs clés	Notes
Purchase	code, supplier, createdBy, items[{product, quantity, buyingPrice, itemTotal}], totalAmount, paidAmount, debts, paymentStatus, paymentMethod	paymentStatus : paid / partial / unpaid
Sale	invoiceNumber (unique), servedBy, items[{product, quantity, sellingPrice, itemTotal}], subtotal, discount, totalAmount, deliveryfees, paymentMethod, paymentStatus, paidAmount, remainAmount, requiresDelivery, deliveryId	paymentStatus : paid / partial / refunded / unpaid ; gère la vente à crédit (remainAmount)
Delivery	sale, status, deliveryMan, deliveryAddress{street,city,phone,notes}, deliveryfees, estimatedArrival, actualArrival	status : pending → preparing → on_route → arrived / failed
CustomerRequest	customerName, customerPhone, product, quantity, status, notes, notifiedAt	status : pending / notified / fulfilled / cancelled — liste d'attente sur rupture de stock

3.4. Finances et stock

Modèle	Champs clés	Notes
Transaction	type (sale/expense/purchase/return), direction (in/out), amount, referenceId, referenceModel, performedBy, date, note	Registre financier unifié, référence polymorphe vers Sale/Expense/Purchase
StockMovement	product, createdBy, type (purchase/sale/return/adjustment), quantity, quantityBefore, quantityAfter, referenceModel, referenceId, note	Journal d'audit des variations de stock
Expense / ExpenseType	title, amount, expenseType, description, date, paidBy	ExpenseType peut avoir un montant fixe (isPriceFixed)
FactureEmploi	employee, salaire, mois, annee, avance, reste, TotalVerser	Bulletin de paie mensuel d'un employé

4. Authentication — middleware Protect

L'authentification repose sur un JWT signé et stocké dans un cookie httpOnly nommé jwt (sameSite: strict, secure activé en production). La connexion se fait par nom d'utilisateur (name de l'employé) et mot de passe, comparés via bcrypt.compare.

```
export const Protect = catchAsync(async (req, res, next) => {
  let token;
  if (req?.cookies?.jwt) {
    token = req.cookies.jwt;
  } else if (req?.headers?.authorization?.startsWith("Bearer ")) {
    token = req.headers.authorization.split(" ")[1];
  }
  if (!token) return next(new APPError("accès refusé", 401));

  const decodedToken = jwt.verify(token, process.env.SecureTokenKey);
  const userexist = await Employee.findById(decodedToken.id)
    .select("+isActive")
    .populate("mission", "name");
  if (!userexist || !userexist.isActive) {
    return next(new APPError("l'accès de l'utilisateur a été désactiver", 401));
  }
  const TokenValide = await userexist.isPaswordchnageAfterToekn(decodedToken.iat);
  if (TokenValide) return next(new APPError("ce accès été désactiver", 401));

  req.user = userexist;
  next();
});
```

Le middleware lit le jeton indifféremment depuis le cookie (client web) ou l'en-tête Authorization: Bearer (client externe potentiel). Il vérifie ensuite trois conditions : validité de la signature JWT, compte employé actif (isActive), et absence de changement de mot de passe postérieur à l'émission du jeton (invalidation implicite des anciens jetons).

Le champ role du modèle User (admin / employee / stock_manager) pose la base d'une différenciation des droits par profil. La mise en œuvre du contrôle d'accès associé — un middleware d'autorisation venant compléter Protect — constitue la prochaine étape prévue pour renforcer la sécurité applicative avant la soutenance.

5. Gestion des erreurs asynchrones

Toutes les fonctions contrôleur sont enveloppées dans `catchAsync`, qui délègue toute exception à `next()`. Une variante `transactional()` encapsule en plus une transaction Mongoose (`session.withTransaction()`), utile pour les opérations qui doivent rester atomiques (par exemple une vente qui décrémente le stock et crée un mouvement de stock en une seule opération logique).

```
export const catchAsync = (fn) => (req, res, next) => {
  fn(req, res, next).catch(next);
};

export const transactional = (fn) =>
  catchAsync(async (req, res, next) => {
    const session = await mongoose.startSession();
    try {
      await session.withTransaction(async () => {
        await fn(req, res, next, session);
      });
    } finally {
      session.endSession();
    }
  });
```

Le gestionnaire d'erreur global distingue explicitement plusieurs types d'erreurs (`JsonWebTokenError` / `TokenExpiredError` → 401, `MulterError` avec message dédié pour dépassement de taille d'image, `ValidationError` Mongoose → 400 avec liste des messages), ce qui constitue une amélioration par rapport à une gestion d'erreurs uniforme à un seul code HTTP.

6. Variables d'environnement

Variable	Rôle
<code>DataBase</code>	URI de connexion MongoDB
<code>PORT</code>	Port HTTP du serveur
<code>SecureTokenKey</code>	Clé de signature/vérification des JWT
<code>tOKENeXPIRE</code>	Durée de validité du token JWT
<code>CookieseXPIRE</code>	Durée de validité du cookie (en jours)
<code>envirement</code>	Bascule production (active les cookies secure)

Remarque : aucun fichier `.env.example` n'est actuellement committé dans le dépôt — contrairement aux bonnes pratiques, la configuration requise n'est donc pas auto-documentée pour un nouvel environnement de déploiement.

VIII. Frontend Web

1. Point d'entrée et bootstrap d'authentification

main.jsx monte <App /> à l'intérieur de StrictMode, avec trois providers globaux : QueryClientProvider (cache TanStack Query), ThemeProvider (thème MUI personnalisé) et DataProvider (contexte React applicatif). Un composant Toaster (react-hot-toast) est monté au niveau racine pour les notifications.

AppRouter.jsx définit un composant RederirectLogin qui interroge GET /employees/me via React Query : en cas de succès (session active), l'utilisateur est redirigé vers l'accueil ; en cas d'erreur (401), la page de connexion s'affiche. Ce mécanisme joue le rôle de garde d'authentification à l'entrée de l'application.

2. Table des routes

Chemin	Composant	Domaine
/login	LoginPage	Authentification
/	HomePage (mise en page + Sidebar)	Accueil
/products	ProductsPage	Catalogue produits
/suppliers	SuppliersPage	Fournisseurs
/employees	EmployeesPage	Employés
/purchases	PurchasePage	Achats
/stock	StockPage	Mouvements de stock
/sales	SalesPage	Ventes / facturation
/delivery	DeliveryPage	Livraisons
/requests	CustomersPage	Demandes clients
/finance	TransactionsPage	Transactions financières
/settings	SettingPage	Types produit/emploi, profil

Les routes métiers sont déclarées comme enfants de la route racine « / » (HomePage), qui joue le rôle de mise en page commune (Sidebar + NavBar + Outlet). Il n'existe pas de composant de garde dédié par route (type ProtectedRoute) : la protection s'appuie uniquement sur RederirectLogin au niveau /login et sur les réponses 401 de l'API.

3. Couche API — services et intercepteurs Axios

Toutes les requêtes HTTP passent par une instance Axios unique, configurée dans `Servises/axiosInstance.js` :

```
export const axiosInstance = axios.create({
  baseURL: import.meta.env.VITE_BASE_URL,
  withCredentials: true, // envoi automatique du cookie httpOnly
});

axiosInstance.interceptors.response.use(
  (response) => response,
  async (error) => {
    if (error?.response?.status === 401 && window.location.pathname !== "/login") {
      window.location.href = "/login";
    }
    return Promise.reject(error);
  },
);
```

L'intercepteur de réponse redirige automatiquement vers `/login` sur toute erreur 401, quel que soit l'écran courant — ce qui centralise la déconnexion forcée en un seul endroit. Un module de service dédié existe par ressource métier (`Products.js`, `Sales.js`, `Purchases.js`, `Employees.js`, `Suppliers.js`, `Delivery.js`, `Customers.js`, `Transactions.js`, `Expenses.js`, `StockMovements.js`, `Users.js`, `EmployeeTypes.js`, `ExpenseTypes.js`, `ProductCategories.js`, `Autontification.js`), chacun encapsulant les appels REST correspondants.

4. Organisation des écrans par domaine

Chaque domaine métier suit une convention de dossier homogène : une page liste (ex. `ProductsPage`, `SalesPage`), un tableau de données associé (via `@tanstack/react-table`), et un sous-dossier `Actions/` regroupant les formulaires d'ajout, de modification et de suppression (`Add.jsx`, `Update.jsx`, `Delete.jsx`).

Domaine	Page	Particularités
Produits	<code>ProductPage</code> / <code>ProductTable</code> / <code>ProductsStatic</code>	Filtrage (<code>Filter.jsx</code>), statistiques produits
Fournisseurs	<code>SuppliersPage</code>	CRUD standard

Domaine	Page	Particularités
Employés	EmployeesPage	Réinitialisation de mot de passe (PasswordReset.jsx)
Achats	PurchasePage / PurchaseTable	CRUD standard
Stock	StockPage / StockTable	Ajout/consultation des mouvements
Ventes	SalesPage / SalesTable	Formulaire multi-étapes (produit, client, paiement, facture)
Livraisons	DeliveryPage / DeliveryTable	Suivi de statut de livraison
Demandes clients	CustomersPage / CustomersTable	Liste d'attente produit
Transactions	TransactionsPage	Vue consolidée des flux financiers
Paramètres	SettingPage (EmploisType, ProductType, Profil)	Référentiels (types) + profil employé

5. Écran de vente — formulaire composite

L'écran de création de vente (Sales/Actions/AddSelleParts/) illustre la complexité métier la plus élevée du frontend : il se décompose en quatre sous-composants assemblés — ProductPart (sélection des articles et quantités), CustomerPart (informations client optionnelles pour la livraison), PaymentPart (mode de paiement, montant payé, calcul du reste dû) et FactureSell / SellFacture (génération et affichage de la facture).

IX

IX. Flux de données — scénarios clés

1. Connexion d'un employé

- L'utilisateur saisit son nom d'utilisateur et son mot de passe dans LoginPage.
- Le frontend envoie POST /api/auth/login avec { username, password }.
- Côté backend, Employee.findOne({ name: username }) récupère l'employé (avec le mot de passe, normalement exclu par select:false), puis employee.matchPassword(password) compare le hash bcrypt.
- Le compte doit être actif (isActive) sinon la connexion est refusée.

- Un JWT signé { id } est émis et posé dans un cookie httpOnly « jwt » (sameSite strict, secure en production).
- Le frontend interroge ensuite GET /employees/me pour récupérer le profil et confirmer la session (via React Query, déclenché par RederirectLogin).

2. Création d'une vente

- Sur SalesPage, l'utilisateur ouvre le formulaire multi-étapes AddSelleParts (produits, client, paiement).
- Le formulaire agrège les articles sélectionnés (product, quantity, sellingPrice) et calcule le sous-total, la remise éventuelle et le montant total.
- La requête POST /api/sales/ crée le document Sale, avec calcul de itemTotal par article, du paidAmount et du remainAmount (vente à crédit possible).
- Si requiresDelivery est vrai, une livraison associée peut être créée (POST /api/delivery/) référencée sur la vente.
- Le stock des produits vendus doit être décrémenté et un mouvement de stock de type « sale » enregistré (StockMovement) pour assurer la traçabilité — logique généralement encapsulée dans une transaction Mongo via le wrapper transactional().
- Une transaction financière de type « sale », direction « in », est créée dans le registre Transaction pour le suivi de trésorerie.

3. Réception d'un achat fournisseur

- Sur PurchasePage, l'utilisateur sélectionne un fournisseur et les produits commandés avec leurs quantités et prix d'achat.
- POST /api/purchases/ crée le document Purchase (items, totalAmount, paidAmount, debts, paymentStatus).
- Le stock des produits reçus est incrémenté et un StockMovement de type « purchase » est journalisé.

- Une transaction financière de type « purchase », direction « out », est enregistrée.

4. Rupture de stock et demande client

- Lorsqu'un produit est en rupture ($\text{quantity} \leq \text{minStockAlert}$ ou 0), le personnel peut créer une CustomerRequest (nom, téléphone, produit, quantité souhaitée), statut initial « pending ».
- Lorsque le produit est réapprovisionné, la demande peut être marquée « notified » (avec horodatage notifiedAt), puis « fulfilled » une fois la vente réalisée.

x

X. Schéma de la base de données

Le modèle de données comporte seize collections MongoDB. Les relations sont matérialisées par des ObjectId typés avec l'option ref côté Mongoose, ce qui permet de peupler les documents liés via populate().

1. Relations principales

- Category → Product : 1:N. Une catégorie regroupe plusieurs produits.
- Supplier → Product : 1:N (optionnelle, conditionnée par hasSupplier). Supplier → productTypes : N:N vers Category.
- EmployeeType → Employee : 1:N via le champ mission.
- Employee → FactureEmploi : 1:N. Un employé a plusieurs bulletins de paie mensuels.
- Supplier / Employee → Purchase : 1:N. Purchase.items → Product : N:N via un tableau de sous-documents.
- Employee (servedBy) → Sale : 1:N. Sale.items → Product : N:N via un tableau de sous-documents.
- Sale → Delivery : 1:1 (optionnelle, conditionnée par requiresDelivery). Employee (deliveryMan) → Delivery : 1:N.
- Product → CustomerRequest : 1:N. Un produit peut avoir plusieurs demandes en attente.

- Product / Employee (createdBy) → StockMovement : 1:N chacun. StockMovement référence polymorphiquement Sale ou Purchase (referenceModel + referenceId).
- ExpenseType → Expense : 1:N. User (paidBy) → Expense : 1:N.
- Transaction référence polymorphiquement Sale, Expense ou Purchase (referenceModel + referenceId), et User (performedBy) → Transaction : 1:N.

2. Index automatiques

Mongoose crée un index primaire sur `_id` pour chaque collection. Les champs marqués unique (Employee.name, Employee.email, User.phone, Sale.invoiceNumber) génèrent des index uniques. Aucun index composé personnalisé n'est actuellement défini ; des requêtes fréquentes comme le filtrage des mouvements de stock par produit et par date, ou des ventes par période, bénéficieraient d'index composés dans une itération future.

XI

XI. Build, exécution et déploiement

1. Backend

```
cd Back-end
npm install
# Créer un fichier .env avec les variables listées ci-dessus
npm start          # ou : nodemon Index.js (si configuré en dev)
```

Contenu attendu du fichier `.env` (à documenter — actuellement non fourni dans le dépôt) :

```
DataBase=mongodb://localhost:27017/bestmark
PORT=5000
SecureTokenKey=une-cle-secrete-aleatoire-suffisamment-longue
tOKENEXPIRE=30d
CookieEXPIRE=30
envirement=development
```

2. Frontend web

```
cd Front-end
npm install
# Créer un fichier .env contenant : VITE_BASE_URL=http://localhost:5000/api
npm run dev      # Démarre le serveur Vite
npm run build    # Build de production dans dist/
npm run preview  # Prévisualise le build de production
```

Le CORS backend est actuellement codé en dur sur `http://localhost:5173` (origine Vite par défaut) ; toute autre origine (déploiement) nécessitera une mise à jour de cette configuration ou son externalisation en variable d'environnement.

XII

XII. Couverture des fonctionnalités vs spécification

Le projet n'est pas encore finalisé au moment de la rédaction de ce rapport. Le tableau suivant reflète honnêtement l'état d'avancement constaté dans le code au 1er septembre 2026.

Fonctionnalité (spec)	Statut	Emplacement
Authentification employé (login, JWT, cookie httpOnly)	Implémenté	Emplye/AuthEmployee.js, Index.js
Gestion des produits et catégories (CRUD + image)	Implémenté	Products/*
Gestion des fournisseurs (CRUD)	Implémenté	Supplieres/*
Achats fournisseurs (CRUD)	Implémenté	Purchases/*
Ventes / facturation (items, paiement, crédit)	Implémenté	sales/*, Sales/Actions/AddSelleParts/*
Livraisons liées aux ventes	Implémenté	Delivery/*
Demandes clients (liste d'attente)	Implémenté	Customers/*
Gestion des employés (CRUD, activation, mot de passe)	Implémenté	Employes/Emplye/*
Bulletins de paie (FactureEmploi)	Modèle présent — couverture routeur/UI à vérifier	Employes/FactureEmployer /
Mouvements de stock (journal d'audit)	Implémenté	stockMovements/*

Fonctionnalité (spec)	Statut	Emplacement
Dépenses et types de dépense	Implémenté	expenses/*
Registre financier centralisé (transactions)	Implémenté	Transactions/*
Autorisation par rôle côté serveur (admin/employee/stock_manager)	Prévue — champ role déjà modélisé, mise en œuvre à venir	Users/User.js
Limitation de débit (anti-bruteforce)	Codée mais désactivée (commentée)	Index.js
Documentation de configuration (.env.example)	Absente	Back-end/
Tests automatisés	Non implémenté	—

13 XIII. Perspectives d'évolution

Au-delà des améliorations techniques identifiées au fil de ce rapport (autorisation par rôle, limitation de débit, tests automatisés), cette partie propose une réflexion à plus long terme sur les évolutions possibles du produit BESTMARK, si la mission devait se poursuivre au-delà du cadre scolaire actuel.

1. Extension multi-magasins

La version actuelle de BESTMARK a été conçue pour un magasin unique. Une évolution naturelle consisterait à introduire la notion d'établissement (Store) dans le modèle de données, avec un rattachement de chaque produit, vente, employé et mouvement de stock à un magasin donné. Cela permettrait à une chaîne de plusieurs points de vente de mutualiser un même back-office tout en conservant des stocks et des équipes distincts par établissement, avec des tableaux de bord consolidés au niveau du réseau.

2. Application mobile pour la gestion de stock

À l'image de ce qui a pu être observé sur d'autres projets de gestion (applications de type check-in par QR code), une application mobile dédiée au personnel de réserve permettrait de scanner un code-barres produit pour consulter ou ajuster directement sa quantité en stock, sans repasser par un poste fixe. L'API

REST actuelle, déjà pensée pour accepter un jeton via l'en-tête Authorization: Bearer en plus du cookie httpOnly, offre une base technique directement réutilisable pour un tel client mobile.

3. Tableau de bord analytique avancé

Le compteur Number_of_sales par produit et le registre Transaction centralisé constituent déjà une base de données exploitable pour des statistiques plus poussées que le tableau de bord actuel : évolution du chiffre d'affaires par période, marge réelle par catégorie de produits (à partir de l'écart entre buyingPrice et sellingPrice), identification des produits à rotation lente pour ajuster les commandes fournisseurs, ou encore suivi de la ponctualité des livraisons. Ces indicateurs pourraient être présentés sous forme de graphiques dans un nouvel écran d'analyse, sans nécessiter de modification du modèle de données existant.

4. Notifications automatiques

Deux mécanismes du système actuel gagneraient à être complétés par des notifications automatiques plutôt que par une consultation manuelle : l'alerte de rupture de stock (basée sur minStockAlert), qui pourrait déclencher un e-mail ou une notification au responsable des achats dès le franchissement du seuil, et le passage d'une demande client au statut « notifié », qui pourrait déclencher automatiquement un SMS ou un e-mail au client concerné plutôt que de dépendre d'une action manuelle du personnel.

5. Vitrine client et prise de commande en ligne

Le périmètre actuel du projet exclut volontairement une interface publique destinée aux clients finaux (cf. chapitre I, section 5). Une évolution ambitieuse consisterait à ouvrir une partie du catalogue produits (déjà modélisé et exposé par l'API) à une vitrine en ligne, permettant aux clients de passer commande directement, la commande alimentant alors le module Ventes existant avec un statut initial « en attente de préparation » plutôt que d'être créée manuellement par le personnel.

6. Synthèse des priorités d'évolution

Le tableau suivant propose une priorisation indicative des évolutions évoquées dans ce chapitre, croisée avec les axes d'amélioration technique déjà identifiés au fil du rapport, pour donner une feuille de route lisible si le projet devait se poursuivre au-delà de la mission actuelle.

Évolution	Priorité	Effort estimé	Bénéfice attendu
Autorisation par rôle côté serveur	Haute	Faible	Sécurité renforcée, prérequis à toute ouverture du système
Activation du rate limiting	Haute	Très faible	Protection anti-bruteforce, déjà codée
Tests automatisés (backend prioritaire)	Haute	Moyen	Fiabilité des évolutions futures
Notifications automatiques (rupture, demandes clients)	Moyenne	Moyen	Réactivité opérationnelle accrue
Tableau de bord analytique avancé	Moyenne	Moyen	Meilleur pilotage de la marge et des stocks
Application mobile de gestion de stock	Basse	Élevé	Autonomie du personnel de réserve
Extension multi-magasins	Basse	Élevé	Passage à l'échelle vers plusieurs points de vente
Vitrine client en ligne	Basse	Élevé	Nouveau canal de vente, hors périmètre initial

Ce glossaire rassemble les principaux termes techniques employés dans ce rapport, à destination d'un lecteur qui ne serait pas familier avec l'ensemble du vocabulaire du développement web backend.

Terme	Définition
API REST	Interface de programmation qui expose des ressources (produits, ventes...) via des adresses (endpoints) et des méthodes HTTP standard (GET, POST, PUT/PATCH, DELETE).
JWT (JSON Web Token)	Jeton signé numériquement, utilisé pour authentifier un utilisateur sans avoir à stocker sa session côté serveur.
Cookie httpOnly	Cookie inaccessible depuis le JavaScript exécuté dans le navigateur, ce qui protège le jeton d'authentification contre le vol par une faille de type XSS.

Terme	Définition
Middleware	Fonction qui s'exécute entre la réception d'une requête HTTP et son traitement final, utilisée ici pour l'authentification, la sécurité ou la gestion des erreurs.
Mongoose	Bibliothèque Node.js qui structure l'accès à une base MongoDB à travers des schémas et des modèles de données typés.
ObjectId	Identifiant unique généré par MongoDB pour chaque document, utilisé pour créer des relations entre collections.
populate()	Fonctionnalité Mongoose qui remplace une référence ObjectId par le document complet qu'elle désigne, lors d'une lecture en base.
Référence polymorphe	Relation dont la collection cible n'est pas fixée à l'avance mais déterminée dynamiquement par un champ (ici referenceModel), utilisée pour le registre financier centralisé.
bcrypt	Algorithme de hachage utilisé pour transformer un mot de passe en une empreinte non réversible avant stockage en base.
Injection NoSQL	Technique d'attaque consistant à injecter des opérateurs de requête MongoDB (comme \$gt ou \$ne) dans une entrée utilisateur pour détourner la logique d'une requête.
CORS	Mécanisme HTTP qui contrôle quelles origines (domaines) sont autorisées à interroger l'API depuis un navigateur.
Endpoint	Adresse précise de l'API (méthode + chemin) correspondant à une action possible, par exemple POST /api/sales/ pour créer une vente.
Contrôleur	Fonction backend qui contient la logique métier associée à un endpoint donné (validation, accès base de données, réponse).
catchAsync	Fonction utilitaire développée pour ce projet, qui évite de répéter un bloc try/catch dans chaque contrôleur asynchrone.
React Query	Bibliothèque frontend qui gère la récupération, la mise en cache et la synchronisation des données provenant de l'API, côté React.
Helmet	Middleware Express qui configure automatiquement un ensemble d'en-têtes HTTP de sécurité recommandés (protection contre le clickjacking, etc.).
HPP (HTTP Parameter Pollution)	Vulnérabilité exploitant l'envoi de plusieurs valeurs pour un même paramètre HTTP, contrée par le middleware hpp.
Transaction Mongoose	Mécanisme garantissant qu'un ensemble d'opérations sur la base de données réussissent ou échouent toutes ensemble, utilisé pour les enchaînements critiques (vente + stock + finance).
Statut de paiement	Champ métier (payé, partiel, remboursé, impayé) utilisé sur les ventes et les achats pour refléter l'état réel du règlement d'une opération.
Slug	Version d'un texte simplifiée pour être utilisée dans une URL (minuscules, sans espaces ni accents), utilisée ici pour les catégories de produits.

1. Exemple de gestion d'erreur

Extrait du gestionnaire d'erreur global, illustrant la différenciation par type d'erreur :

```
if (err.name === 'JsonWebTokenError' || err.name === "TokenExpiredError") {
  return res.status(401).json({ status: 'error', message: 'accès refusé' });
}
if (err.name === 'MulterError') {
  return res.status(400).json({
    status: 'fail',
    message: err.code === 'LIMIT_FILE_SIZE'
      ? "L'image dépasse la taille maximale autorisée (5MB)"
      : err.message,
  });
}
if (err.name === 'ValidationError') {
  const errors = Object.values(err.errors).map((error) => error.message);
  res.status(400).send(errors);
}
```

2. Améliorations prévues

Activer et ajuster le limiteur de débit `express-rate-limit`, en particulier sur la route de connexion. Le code est déjà écrit dans `Index.js` mais reste commenté ; son activation, associée à un seuil plus strict sur `/api/auth/login` que sur le reste de l'API, renforcerait la protection contre les attaques par force brute sans effort de développement supplémentaire significatif.

Implémenter un middleware d'autorisation par rôle sur les routes sensibles. Le champ `role` du modèle `User` (`admin` / `employee` / `stock_manager`) est déjà modélisé ; il manque le middleware qui, à partir du rôle de l'utilisateur authentifié, restreint l'accès à certaines routes (suppression d'un employé, consultation des transactions financières) aux seuls profils autorisés.

Committer un fichier `.env.example` documentant chaque variable d'environnement requise (DataBase, PORT, SecureTokenKey, tOKENeXPIRE, CookieseXPIRE, envirement), sans exposer de valeurs réelles, afin qu'un nouveau développeur ou un environnement de déploiement puisse configurer le projet sans avoir à deviner les noms de variables attendus.

Externaliser l'origine CORS, actuellement codée en dur sur `http://localhost:5173`, en variable d'environnement. Cette évolution simple est indispensable avant tout déploiement en production, l'origine du frontend n'étant alors plus localhost.

Ajouter des index composés MongoDB sur les collections à forte volumétrie de requêtes filtrées, en particulier { MemberId: 1, CheckIn: -1 } sur StockMovement (filtrage par produit et par date) et un index équivalent sur Sale (filtrage par période), pour anticiper la dégradation des performances à mesure que l'historique de ventes grandit.

Finaliser la couverture du sous-module FactureEmploi côté routeur et interface, actuellement limité au modèle de données, pour que la gestion de la paie soit aussi complète côté utilisateur que les autres modules du projet.

Écrire des tests automatisés, backend (Jest) et frontend (Vitest), en priorité sur les modules les plus critiques identifiés dans ce rapport : authentification, calcul du montant restant dû sur une vente à crédit, et enchaînement stock/transaction lors d'un achat ou d'une vente.

3. Bilan personnel et conclusion générale

BESTMARK couvre la majorité des besoins exprimés dans la fiche de mission professionnelle : gestion des produits, des commandes (achats et ventes) et des clients, avec un suivi financier et un journal de stock qui dépassent le périmètre minimal initial. L'architecture en deux applications (API REST + client React), organisée par domaine métier de façon cohérente sur les deux couches, facilite la lecture et l'évolution du code.

Sur le plan personnel, cette mission de trois mois a été l'occasion de mener seul un projet de bout en bout, depuis la modélisation des données jusqu'à l'interface utilisateur, en passant par la sécurisation de l'API. L'usage systématique de checklists Notion par module métier a été une méthode de travail efficace pour ne pas perdre de vue les règles métier au fil du développement, et c'est une pratique que je compte conserver pour mes futurs projets. Les apprentissages les plus marquants concernent la sécurité backend et la gestion centralisée des erreurs, deux sujets abordés jusque-là de façon superficielle et que ce projet m'a permis d'approfondir concrètement, en les mettant en œuvre sur un cas réel plutôt qu'en théorie.

Les principaux axes d'amélioration identifiés concernent la sécurité (autorisation par rôle, limitation de débit à activer), la documentation d'exploitation (variables d'environnement) et les tests automatisés. Le projet étant encore en cours de finalisation, ces points constituent la feuille de route pour les prochaines itérations avant la soutenance.

D'un point de vue technique, ce projet a également renforcé ma compréhension de ce que signifie réellement « développer en autonomie » sur une durée de plusieurs mois : au-delà d'écrire du code fonctionnel, il s'agit de savoir prioriser (quel module développer en premier, quelle checklist respecter avant de passer à la suite), documenter ses choix pour pouvoir les expliquer a posteriori, et accepter qu'un projet de cette ampleur ne soit jamais totalement terminé à un instant donné, mais toujours perfectible — d'où l'importance donnée, dans ce rapport, à un chapitre dédié aux perspectives d'évolution plutôt qu'à une simple liste de bugs restants.

Je retiens enfin de cette mission une meilleure capacité à évaluer, avant de coder, l'impact d'un choix de conception sur l'ensemble du système : décider que le stock ne devait jamais être modifié directement mais toujours via un mouvement de stock tracé, ou que les transactions financières devaient être centralisées plutôt que dispersées dans chaque module, sont des décisions prises tôt dans le projet dont j'ai pu mesurer, au fil des trois mois, à quel point elles ont simplifié le développement des modules suivants. C'est sans doute l'enseignement le plus transférable à mes futurs projets, au-delà des technologies précises utilisées ici.